

Universidad
Tecnológica
de Pereira

Facultad de Ingenierías

Ingeniería de Sistemas y Computación

IS453

ProgIIIG1

Actividad03 : Casos con Listas

Andrés David Rincón Salazar
andres.rincon2@utp.edu.co
ID: 11.289.05.513

Pereira, Colombia

- Octubre 2026 -

Índice general

1. Introducción **2**

2. Punto 1: Familia Simpson **4**
2.1. Cambio de representación 4
2.2. Código 5
2.3. Árbol SLD 9

3. Punto 2: Ciudades de Canadá **11**
3.1. Cambio de representación 11
3.2. Código 12
3.3. Árboles SLD 16
3.3.1. Versión segura (árbol completo) 16
3.3.2. Versión ingenua (árbol infinito) 16

4. Repositorio **20**

Capítulo 1

Introducción

Esta actividad retoma dos ejercicios ya resueltos con hechos simples y los reescribe usando **estructuras** (términos compuestos) y **listas**. Las reglas se modificaron para que, aun con la nueva representación de los datos, el programa resuelva exactamente las mismas consultas:

- **Punto 1:** familia Simpson (numeral 1 de la Actividad 1). Los hechos `padre_de/2`, `madre_de/2`, `hombre/1` y `mujer/1` se reemplazan por las estructuras `persona(Nombre, Sexo)` y `familia(Progenitores, Hijos)`, donde los dos argumentos de `familia/2` son listas.
- **Punto 2:** ciudades de Canadá (punto 3 de la Actividad 2). Los hechos `arista(Origen, Destino, Costo)` se reemplazan por una lista de adyacencia por ciudad, `nodo(Ciudad, [enlace(Destino, Costo), ...])`.

Cada programa conserva su encabezado descriptivo (propósito, predicados y salvedades), comentarios de sección y un bloque de consultas de validación.

Verificación de equivalencia

Que "siga resolviendo las mismas consultas" se comprobó de forma automática y no solo con las consultas del encabezado. Se cargaron el programa original y el nuevo en SWI-Prolog 10.0.2 y se compararon sus respuestas:

- **Simpson:** para cada uno de los 11 predicados de parentesco se probaron los 13×13 pares de individuos, y se comparó además la enumeración completa con ambos argumentos sin instanciar y el sexo de cada individuo. Los conjuntos de respuestas son idénticos. El orden de las soluciones sí puede cambiar, porque ahora se recorren listas y no hechos sueltos.
- **Canadá:** se compararon, con el orden de las soluciones, `conexion_directa/3`, `conectado_con/4`, `tiene_aristas/1`, `camino/4`, `hay_camino/2` y `costo_via/4` sobre todos los pares y ternas de las seis ciudades más Toronto. Las respuestas son idénticas, también en orden, porque las listas de adyacencia conservan el orden de los hechos originales.

Árboles de decisión (SLD)

Como en la Actividad 2, al final de cada punto se incluye el árbol SLD de su consulta principal, generado con el paquete *SLDNF Draw* de SWI-Prolog (`pack_install(sldnfdraw)`) a partir de las mismas cláusulas del programa y dibujado con el paquete `forest` de LaTeX. Los archivos de entrada y el script que los regenera están en `arboles/fuentes`. Para que la herramienta pueda dibujarlos, en esos archivos `\+` se escribe `not/1` (es la misma negación por fallo), las variables anónimas `_` llevan nombre y `member/2` se escribe con sus dos cláusulas estándar. Esto último es deliberado: con listas, el recorrido de `member/2` es justo lo que hay que ver en el árbol. La notación es la de la Actividad 2:

- Cada nodo es el resolvente (los objetivos pendientes); las variables llevan un subíndice que indica en qué paso de resolución fueron renombradas.
- El texto azul entre llaves es la ligadura (sustitución) que produjo la unificación con la cláusula elegida.
- Un recuadro es una negación por fallo: dentro está el subárbol de `memberchk/2` y su resultado.
- **true** (verde) es una rama de éxito, **false** (rojo) una rama fallida y $\cdots$ un subárbol que existe pero no se dibuja.
- Prolog recorre el árbol en profundidad y de izquierda a derecha; el *backtracking* es el paso de una rama a la siguiente rama hermana a la derecha.

Una consecuencia de usar listas: el mismo razonamiento necesita más niveles que con hechos sueltos, porque cada acceso a un hecho de la Actividad 1 o 2 pasa ahora por un hecho `familia/2` o `nodo/2` y por un recorrido de `member/2`. Los árboles son anchos o altos; el PDF es vectorial, así que se pueden ampliar sin perder nitidez.

Capítulo 2

Punto 1: Familia Simpson

2.1. Cambio de representación

Concepto	**Actividad 1 (hechos)**	**Actividad 3 (estructuras y listas)**
Padres e hijos	`padre_de(homero, bart).` `madre_de(marge, bart).`	`familia([homero, marge], [bart, lisa, maggie]).`
Sexo	`hombre(herbert).` y reglas `hombre(N) :- padre_de(N, _).`	`persona(herbert, masculino).`
`padre_de/2`, `madre_de/2`	hechos	reglas: `progenitor_de/2` filtrado por `hombre/1` o `mujer/1`
`progenitor_de/2`	dos cláusulas (padre o madre)	`member/2` sobre las listas de una `familia/2`
Hermanos, abuelos, primos	recorren `progenitor_de/2` de a un hijo	usan `hijos_de/2`, que devuelve la lista de hijos con `findall/3`

Decisiones de diseño:

- El sexo ya no se deduce de "ser padre" o "ser madre": se declara una vez por persona. Por eso `padre_de/2` y `madre_de/2` pasan a ser reglas, y los hechos `hombre/1` y `mujer/1` dejan de estar repartidos entre hechos y reglas (lo que causaba la advertencia de cláusulas no contiguas de la Actividad 1).
- Herbert y Ling tienen un solo progenitor conocido, así que cada uno va en su propio `familia/2` con una lista de un elemento. Meter a Herbert en la familia de Homero haría a Mona su madre, algo que la Actividad 1 nunca afirmó.

- Una persona sin hecho `persona/2` no tiene sexo, y las reglas que lo exigen fallan para ella. Cada individuo nuevo debe registrarse.

2.2. Código

Listing 2.1: ProgIIIG1-Act03-Pto01-Andres_David_Rincon_Salazar.pl

```
/*
================================================================================
Publication Date: 2026-10-10
Publication Time: 21:30
Code Version: 1.0
Author: Ing(c) Andres David Rincon Salazar
Programming Language: Prolog
Language Version: ISO/IEC 13211-1 (SWI-Prolog 10.0.2)
Compiler: SWI-Prolog 10.0.2
Operating System: Windows 11 - 24H2
Presented to: Ramiro Andres Barrios Valencia
University: Universidad Tecnologica de Pereira
Program: Ingenieria de Sistemas y Computacion
Based on: ProgIIIG1-Act01-Pto01 (Simpson family, facts only)

Program Description:
Simpson family tree rewritten with structures and lists. The one-fact-per-
relationship model of Activity 1 (padre_de/2, madre_de/2, hombre/1, mujer/1)
is replaced by two kinds of facts:
  - persona(Nombre, Sexo): structure with the sex of each individual.
  - familia(Progenitores, Hijos): structure with two lists, the parents and
    the children of one household.
Every relationship of Activity 1 is still answered with the same predicate
names and arguments, but now it is a rule that traverses those lists with
member/2 and findall/3. Validation queries are included at the end of the file.

Predicates description:
- persona(Nombre, Sexo): (fact) Sexo is masculino or femenino.
- familia(Progenitores, Hijos): (fact) lists of parents and of children.
- hombre/1, mujer/1: (rules) sex of an individual, read from persona/2.
- progenitor_de(Progenitor, Hijo): (rule) member of a Progenitores list that
  has Hijo in the Hijos list of the same familia.
- hijos_de(Progenitor, Hijos): (rule) list of all children of Progenitor.
- padre_de/2, madre_de/2: (rules) progenitor_de/2 filtered by sex.
- es_hijo(Hijo, Padre): (rule) true when Hijo is child of Padre.
- hermano_de/2, hermana_de/2: (rules) siblings sharing one progenitor.
- abuelo_de/2, abuela_de/2: (rules) grandparents (more than one generation).
- tio_de/2, tia_de/2: (rules) uncle/aunt: sibling of a progenitor.
- primo_de/2, prima_de/2: (rules) cousins; sex checked on 1st argument.

Warnings (Salvedades):
- Herbert and Ling have a single known parent, so they are listed in their
  own familia/2 with one progenitor. Putting Herbert in the same familia
```

```
  as Homero would make Mona his mother, which Activity 1 never stated.
- A person with no persona/2 fact has no sex: padre_de/2, madre_de/2, and
  every rule that checks hombre/1 or mujer/1 fail for him or her.
- Some queries may repeat solutions (siblings share two parents).
- primo_de/2 and prima_de/2 only check the sex of the first argument.
================================================================================
*/

%================================================================================
% FACTS: structure persona(Nombre, Sexo)
%================================================================================
persona(abraham,    masculino).
persona(mona,       femenino).
persona(homero,     masculino).
persona(herbert,    masculino).
persona(clancy,     masculino).
persona(jacqueline, femenino).
persona(marge,      femenino).
persona(patty,      femenino).
persona(selma,      femenino).
persona(bart,       masculino).
persona(lisa,       femenino).
persona(maggie,     femenino).
persona(ling,       femenino).

%================================================================================
% FACTS: structure familia(Progenitores, Hijos), both arguments are lists
%================================================================================
familia([abraham, mona],       [homero]).
familia([abraham],             [herbert]).
familia([homero, marge],       [bart, lisa, maggie]).
familia([clancy, jacqueline],  [patty, selma, marge]).
familia([selma],               [ling]).

%================================================================================
% RULES: sex of an individual, read from the persona/2 structure
%================================================================================
hombre(Nombre) :- persona(Nombre, masculino).
mujer(Nombre)  :- persona(Nombre, femenino).

%================================================================================
% RULES: generic parent and list of children
%================================================================================
progenitor_de(Progenitor, Hijo) :-
    familia(Progenitores, Hijos),
    member(Hijo, Hijos),
    member(Progenitor, Progenitores).

hijos_de(Progenitor, Hijos) :-
    findall(Hijo, progenitor_de(Progenitor, Hijo), Hijos).

%================================================================================
```

```
% RULES: father and mother (progenitor filtered by sex)
%=============================================================================
padre_de(Padre, Hijo) :- progenitor_de(Padre, Hijo), hombre(Padre).
madre_de(Madre, Hijo) :- progenitor_de(Madre, Hijo), mujer(Madre).

%=============================================================================
% RULE: child of a parent (es_hijo(Hijo, Padre))
%=============================================================================
es_hijo(Hijo, Padre) :-
    progenitor_de(Padre, Hijo).

%=============================================================================
% RULES: siblings (children of a common progenitor, distinct)
%=============================================================================
hermano_de(Hijo1, Hijo2) :-
    progenitor_de(Padre, Hijo1),
    hijos_de(Padre, Hijos),
    member(Hijo2, Hijos),
    Hijo1 \== Hijo2,
    hombre(Hijo1).

hermana_de(Hijo1, Hijo2) :-
    progenitor_de(Padre, Hijo1),
    hijos_de(Padre, Hijos),
    member(Hijo2, Hijos),
    Hijo1 \== Hijo2,
    mujer(Hijo1).

%=============================================================================
% RULES: multi-generation relationships (grandfather/grandmother)
%=============================================================================
abuelo_de(Abuelo, Nieto) :-
    padre_de(Abuelo, Padre),
    hijos_de(Padre, Nietos),
    member(Nieto, Nietos).

abuela_de(Abuela, Nieto) :-
    madre_de(Abuela, Madre),
    hijos_de(Madre, Nietos),
    member(Nieto, Nietos).

%=============================================================================
% RULES: uncles/aunts (sibling of a parent of the nephew/niece)
%=============================================================================
tio_de(Tio, Sobrino) :-
    progenitor_de(Padre, Sobrino),
    hermano_de(Tio, Padre).

tia_de(Tia, Sobrino) :-
    progenitor_de(Madre, Sobrino),
    hermana_de(Tia, Madre).
```

```
%=============================================================================
% RULES: cousins (children of siblings; sex checked on 1st argument)
%=============================================================================
primo_de(Primo, Primo2) :-
    progenitor_de(Padre1, Primo),
    ( hermano_de(Padre2, Padre1) ; hermana_de(Padre2, Padre1) ),
    hijos_de(Padre2, Primos),
    member(Primo2, Primos),
    hombre(Primo).

prima_de(Prima, Prima2) :-
    progenitor_de(Padre1, Prima),
    ( hermano_de(Padre2, Padre1) ; hermana_de(Padre2, Padre1) ),
    hijos_de(Padre2, Primas),
    member(Prima2, Primas),
    mujer(Prima).

/*
=============================================================================
Validation queries
=============================================================================
?- padre_de(homero, bart).
true.
?- madre_de(marge, lisa).
true.
?- es_hijo(bart, homero).
true.
?- es_hijo(X, homero).
X = bart ;
X = lisa ;
X = maggie.
?- hijos_de(homero, Hijos).
Hijos = [bart, lisa, maggie].
?- abuelo_de(abraham, X).
X = bart ;
X = lisa ;
X = maggie.
?- abuela_de(mona, bart).
true.
?- hermano_de(bart, lisa).
true.
?- hermana_de(lisa, bart).
true.
?- tio_de(herbert, bart).
true.
?- tia_de(patty, bart).
true.
?- primo_de(bart, ling).
true.
?- prima_de(ling, bart).
true.
?- abuelo_de(bart, X).
```

```
false.
================================================================================
*/
```

2.3. Árbol SLD

Consulta dibujada: `padre_de(P, bart)`. Es la consulta que mejor muestra el cambio de representación, porque obliga a recorrer las listas de `familia/2` y a filtrar por sexo con `persona/2`. El árbol se muestra completo (8 niveles) y es finito:

- El primer nivel desarrolla `padre_de/2` en `progenitor_de(P, bart), hombre(P)`, y el segundo escoge un hecho `familia/2`. Hay cinco hechos, así que hay cinco ramas hermanas.
- En cuatro de ellas `member(bart, Hijos)` recorre la lista de hijos hasta llegar a `[]` sin encontrar a Bart, y la rama termina en `false`. Las familias de Herbert y de Ling tienen un solo hijo, por eso esas ramas son cortas.
- En `familia([homero, marge], [bart, lisa, maggie])` Bart es la cabeza de la lista, así que la primera cláusula de `member/2` tiene éxito. Luego `member(P, [homero, marge])` produce `P = homero` y `hombre(homero)` se reduce a `persona(homero, masculino)`: es la única rama de éxito.
- Al retroceder, `P = marge` llega a `persona(marge, masculino)`, que no existe, y falla. Esa es la rama donde el filtro `hombre/1` convierte a `progenitor_de/2` en `padre_de/2`. Después Prolog sigue recorriendo `[lisa, maggie]` en busca de otro `bart`, que no hay.

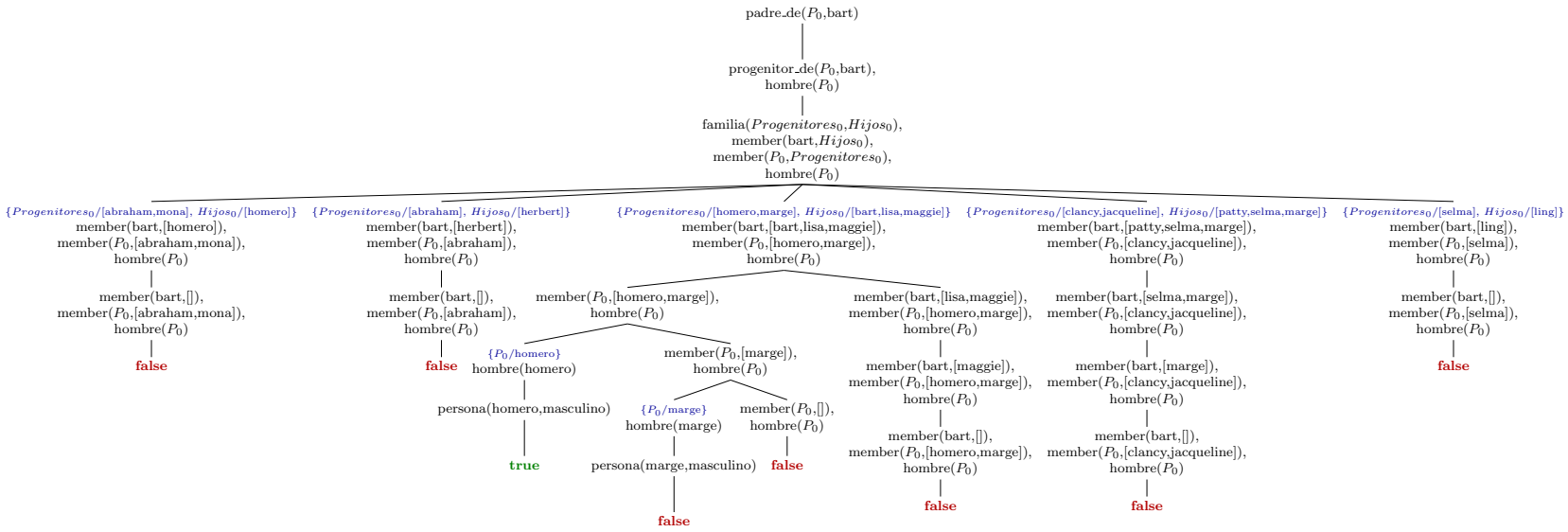


Figura 2.1: Árbol SLD completo de `padre_de(P, bart)`.

Capítulo 3

Punto 2: Ciudades de Canadá

3.1. Cambio de representación

Concepto	**Actividad 2 (hechos)**	**Actividad 3 (estructuras y listas)**
Aristas	nueve hechos `arista(Origen, Destino, Costo).`	seis hechos `nodo(Ciudad, Enlaces)`; `Enlaces` es una lista de estructuras `enlace(Destino, Costo)`
Arista directa	hecho `arista/3`	`conexion_directa/3`: `nodo(A, Es), member(enlace(B, C), Es)`
`conectado_con/4`, `tiene_aristas/1`, `recorrer/5`, `ruta_ingenua/2`	usan `arista/3`	usan `conexion_directa/3`
Vecinos de un nodo	no existía	`vecinos_de/2` devuelve la lista con `findall/3`

Decisiones de diseño:

- El predicado `arista/3` desaparece. Todo acceso a las aristas pasa por `conexion_directa/3`, que es la única regla que recorre las listas; el resto del programa no depende de cómo se guardan.
- Winnipeg no tiene aristas de salida pero sí debe tener su hecho: `nodo(winnipeg, [])`. Si faltara, `tiene_aristas/1` seguiría respondiendo `true` (porque Winnipeg es destino), pero `vecinos_de(winnipeg, V)` fallaría en lugar de responder `V = []`.

- Se conservó `ruta_ingenua/2` porque la Actividad 2 la usa para mostrar que el árbol SLD es infinito por el ciclo Edmonton → Saskatoon → Calgary → Edmonton. La versión segura (`camino/4`, `hay_camino/2`) sigue usando una lista de nodos visitados.

3.2. Código

Listing 3.1: ProgIIIG1-Act03-Pto02-Andres_David_Rincon_Salazar.pl

```
/*
================================================================================
Publication Date: 2026-10-10
Publication Time: 21:45
Code Version: 1.0
Author: Ing(c) Andres David Rincon Salazar
Programming Language: Prolog
Language Version: ISO/IEC 13211-1 (SWI-Prolog 10.0.2)
Compiler: SWI-Prolog 10.0.2
Operating System: Windows 11 - 24H2
Presented to: Ramiro Andres Barrios Valencia
University: Universidad Tecnologica de Pereira
Program: Ingenieria de Sistemas y Computacion
Based on: ProgIIIG1-Act02-Pto03 (Canadian cities, one fact per edge)

Program Description:
Directed weighted graph of Canadian cities (Vancouver, Edmonton, Calgary,
Saskatoon, Regina, Winnipeg) rewritten with structures and lists. The
one-fact-per-edge model arista(Origen, Destino, Costo) of Activity 2 is
replaced by an adjacency list per city:
    nodo(Ciudad, [enlace(Destino, Costo), ...])
The structure enlace/2 holds a neighbour and the cost of reaching it; the
list holds all the edges that leave Ciudad. arista/3 disappears: every rule
now reads the edges by traversing those lists with member/2.
The program still answers the same questions of the slides:
  - Is there a connection between Saskatoon and Vancouver?
  - Which nodes is Regina connected to, and at what cost?
  - Does a node have edges?
  - What is the cost of going from X to Z passing through Y?
  - Is it possible to travel from Edmonton to Calgary?
The last question needs recursion. A naive version (ruta_ingenua/2) shows
how SLD resolution loops on the cycle edmonton -> saskatoon -> calgary ->
edmonton, and a safe version with a list of visited nodes (hay_camino/2)
fixes it. Validation queries are included at the end of the file.

Predicates description:
- nodo(Ciudad, Enlaces): (fact) Ciudad and the list of its outgoing edges.
- conexion_directa(A, B, Costo): (rule) B is in the Enlaces list of A.
- vecinos_de(Nodo, Vecinos): (rule) list of the cities Nodo has an edge to.
- conectado_con(Nodo, Otro, Costo, Sentido): (rule) neighbours of Nodo;
  Sentido is salida (Nodo -> Otro) or entrada (Otro -> Nodo).
```

```
- tiene_aristas(Nodo): (rule) Nodo has at least one incoming/outgoing edge.
- camino(Origen, Destino, Nodos, Costo): (rule) simple directed path.
- hay_camino(Origen, Destino): (rule) safe reachability.
- costo_via(X, Y, Z, Costo): (rule) cost of a path X -> Z that passes by Y.
- ruta_ingenua(Origen, Destino): (rule) naive reachability, may not end.

Warnings (Salvedades):
- Edges are directed: Vancouver reaches Saskatoon but not the other way.
- Costs are the numbers written on the edges of the slide.
- Winnipeg has no outgoing edge, so its list is empty: nodo(winnipeg, []).
  Every city must have its own nodo/2 fact, even with an empty list: a
  city that only appears as a destination inside another list is not
  seen as an origin. A city with no nodo/2 fact (Toronto) has no edges.
- costo_via/4 considers simple paths X -> Z that contain Y as an
  intermediate node; there can be several answers (one per path).
- ruta_ingenua/2 is only a didactic example: it answers true for
  reachable pairs but, because of the cycle, keeps producing more true
  answers forever (one per lap) and, for unreachable pairs, keeps looping
  until it overflows the stack (about one minute with the default 1 GB
  stack limit of SWI-Prolog). Do not use it as a solution.
================================================================================
*/

%================================================================================
% FACTS: adjacency lists, nodo(Ciudad, [enlace(Destino, Costo), ...])
%================================================================================
nodo(vancouver, [enlace(edmonton, 16), enlace(calgary, 13)]).
nodo(edmonton,  [enlace(saskatoon, 12)]).
nodo(calgary,   [enlace(edmonton, 4), enlace(regina, 14)]).
nodo(saskatoon, [enlace(calgary, 9), enlace(winnipeg, 20)]).
nodo(regina,    [enlace(saskatoon, 7), enlace(winnipeg, 4)]).
nodo(winnipeg,  []).

%================================================================================
% RULES: direct connections (questions 1 and 2 of the slides)
%================================================================================
conexion_directa(A, B, Costo) :-
    nodo(A, Enlaces),
    member(enlace(B, Costo), Enlaces).

vecinos_de(Nodo, Vecinos) :-
    nodo(Nodo, Enlaces),
    findall(Vecino, member(enlace(Vecino, _), Enlaces), Vecinos).

conectado_con(Nodo, Otro, Costo, salida)  :- conexion_directa(Nodo, Otro, Costo).
conectado_con(Nodo, Otro, Costo, entrada) :- conexion_directa(Otro, Nodo, Costo).

%================================================================================
% RULE: a node has edges if some edge leaves or reaches it
%================================================================================
tiene_aristas(Nodo) :-
    ( conexion_directa(Nodo, _, _) ; conexion_directa(_, Nodo, _) ), !.
```

```
%=============================================================================
% RULES: safe paths (visited list) -> Edmonton to Calgary question
%=============================================================================
camino(Origen, Destino, Nodos, Costo) :-
    recorrer(Origen, Destino, [Origen], NodosInv, Costo),
    reverse(NodosInv, Nodos).

recorrer(Destino, Destino, Visitados, Visitados, 0).
recorrer(Actual, Destino, Visitados, Ruta, Costo) :-
    conexion_directa(Actual, Siguiente, C1),
    \+ memberchk(Siguiente, Visitados),
    recorrer(Siguiente, Destino, [Siguiente|Visitados], Ruta, C2),
    Costo is C1 + C2.

hay_camino(Origen, Destino) :-
    camino(Origen, Destino, _, _), !.

%=============================================================================
% RULE: cost from X to Z passing through Y (Y is neither X nor Z)
%=============================================================================
costo_via(X, Y, Z, Costo) :-
    camino(X, Z, Nodos, Costo),
    Y \== X,
    Y \== Z,
    memberchk(Y, Nodos).

%=============================================================================
% DIDACTIC RULE: naive reachability (no visited list)
%
% SLD tree for ?- ruta_ingenua(edmonton, calgary).
%
%   ruta_ingenua(edmonton, calgary)
%     |-- clause 1: conexion_directa(edmonton, calgary, _)       FAIL
%     |-- clause 2: conexion_directa(edmonton, Z, _)  Z = saskatoon
%           ruta_ingenua(saskatoon, calgary)
%             |-- clause 1: conexion_directa(saskatoon, calgary, _)  SUCCESS (true)
%             |-- clause 2 (on backtracking): Z = calgary
%                   ruta_ingenua(calgary, calgary)
%                     |-- clause 1: FAIL
%                     |-- clause 2: Z = edmonton -> ruta_ingenua(edmonton, calgary)
%                                   ... the initial goal again: INFINITE LOOP
%=============================================================================
ruta_ingenua(Origen, Destino) :- conexion_directa(Origen, Destino, _).
ruta_ingenua(Origen, Destino) :-
    conexion_directa(Origen, Intermedio, _),
    ruta_ingenua(Intermedio, Destino).

/*
=============================================================================
Validation queries
=============================================================================
```

```
% Question 1: connection between Saskatoon and Vancouver
?- conexion_directa(saskatoon, vancouver, C).
false.
?- hay_camino(saskatoon, vancouver).
false.
?- hay_camino(vancouver, saskatoon).
true.

% Question 2: nodes connected to Regina and cost of each connection
?- conectado_con(regina, Otro, Costo, Sentido).
Otro = saskatoon, Costo = 7,  Sentido = salida ;
Otro = winnipeg,  Costo = 4,  Sentido = salida ;
Otro = calgary,   Costo = 14, Sentido = entrada.
?- vecinos_de(regina, Vecinos).
Vecinos = [saskatoon, winnipeg].

% Rule: node has edges
?- tiene_aristas(regina).
true.
?- tiene_aristas(toronto).
false.

% Rule: cost from X to Z passing through Y
?- costo_via(vancouver, calgary, winnipeg, C).
C = 55 ;
C = 49 ;
C = 54 ;
C = 31 ;
false.
?- costo_via(vancouver, regina, winnipeg, C).
C = 55 ;
C = 54 ;
C = 31 ;
false.
?- costo_via(winnipeg, regina, vancouver, C).
false.

% Most interesting question: Edmonton -> Calgary
?- hay_camino(edmonton, calgary).
true.
?- camino(edmonton, calgary, Nodos, Costo).
Nodos = [edmonton, saskatoon, calgary],
Costo = 21 ;
false.
?- ruta_ingenua(edmonton, calgary).
true ;
true ;    % ... one more true per lap around the cycle: never ends (see SLD tree above)
?- ruta_ingenua(saskatoon, vancouver).
ERROR: Stack limit (1.0Gb) exceeded.    % only after about one minute
================================================================================
*/
```

3.3. Árboles SLD

3.3.1. Versión segura (árbol completo)

Consulta dibujada: `recorrer(edmonton, calgary, [edmonton], R, C)`, que es el núcleo de `camino(edmonton, calgary, Nodos, Costo)`. El árbol se muestra completo y es finito:

- Tiene una sola rama de éxito, con `R = [calgary, saskatoon, edmonton]` y `C = 21`. `camino/4` invierte la lista y responde `[edmonton, saskatoon, calgary]`.
- Cada paso por el grafo se desdobla en tres niveles: `conexion_directa/3`, el hecho `nodo/2` que entrega la lista de enlaces y el recorrido de esa lista con `member/2`. En la Actividad 2 el mismo paso era un solo nivel (`arista/3`). Cuando la lista tiene más de un enlace, la segunda cláusula de `member/2` produce la rama hermana con el resto de la lista (por ejemplo `enlace(regina, 14)` después de `enlace(edmonton, 4)`).
- Al seguir buscando desde Calgary, el enlace hacia Edmonton se poda porque `memberchk(edmonton, ...)` es verdadero: la lista de visitados corta el ciclo que hace infinita a la versión ingenua.
- Las demás ramas terminan en `false`: o vuelven a un nodo ya visitado, o llegan a Winnipeg, cuya lista de enlaces es `[]`, de modo que `member/2` falla de inmediato.

3.3.2. Versión ingenua (árbol infinito)

Consulta dibujada: `ruta_ingenua(edmonton, calgary)`. Este árbol es infinito, así que se corta en el nivel 13, donde el objetivo inicial `ruta_ingenua(edmonton, calgary)` reaparece en la rama izquierda tras una vuelta completa al ciclo `edmonton` → `saskatoon` → `calgary` → `edmonton`:

- La primera cláusula de `ruta_ingenua/2` falla de inmediato (Edmonton no tiene enlace directo a Calgary: su lista es `[enlace(saskatoon, 12)]`). La segunda avanza a Saskatoon.
- En el nivel 8 aparece la primera rama de éxito: `ruta_ingenua(saskatoon, calgary)` se resuelve con `enlace(calgary, 9)`, la cabeza de la lista de Saskatoon. Es el primer `true` que imprime Prolog.
- En el nivel 12 el objetivo inicial vuelve a aparecer. Cada vuelta al ciclo cuesta ahora 12 niveles y no 3 como en la Actividad 2, porque cada arista se desarrolla en `conexion_directa/3`, `nodo/2` y `member/2`; el comportamiento es el mismo: un `true` por vuelta, sin terminar nunca.

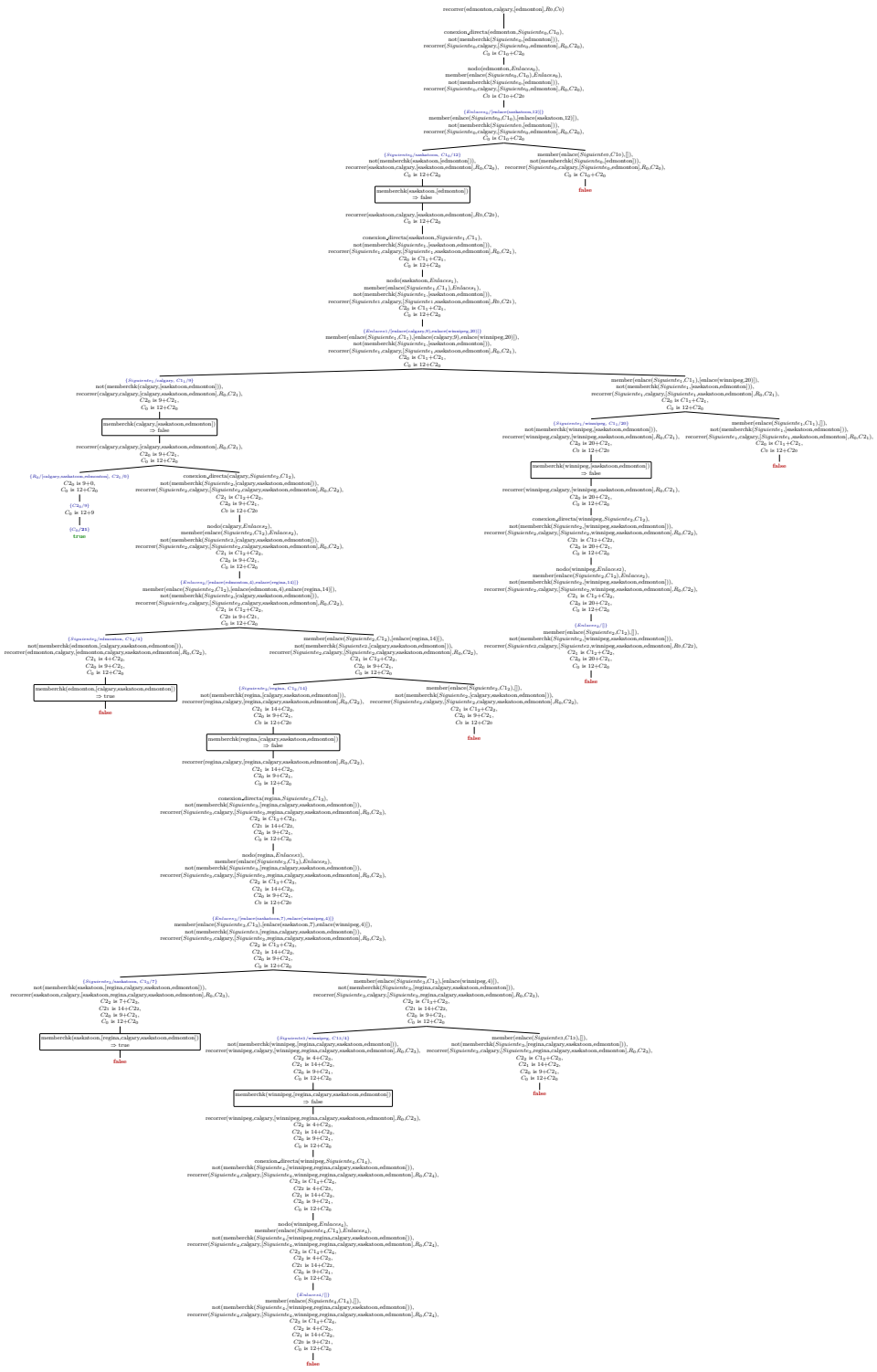


Figura 3.1: Árbol SLD completo de `recorrer(edmonton, calgary, [edmonton], R, C)`.

- Las ramas de la derecha (por Regina y Winnipeg) existen en el árbol, pero Prolog nunca llega a ellas: la búsqueda en profundidad se queda para siempre en la rama infinita de la izquierda. Por eso la estrategia de búsqueda de Prolog (en profundidad) no es completa.

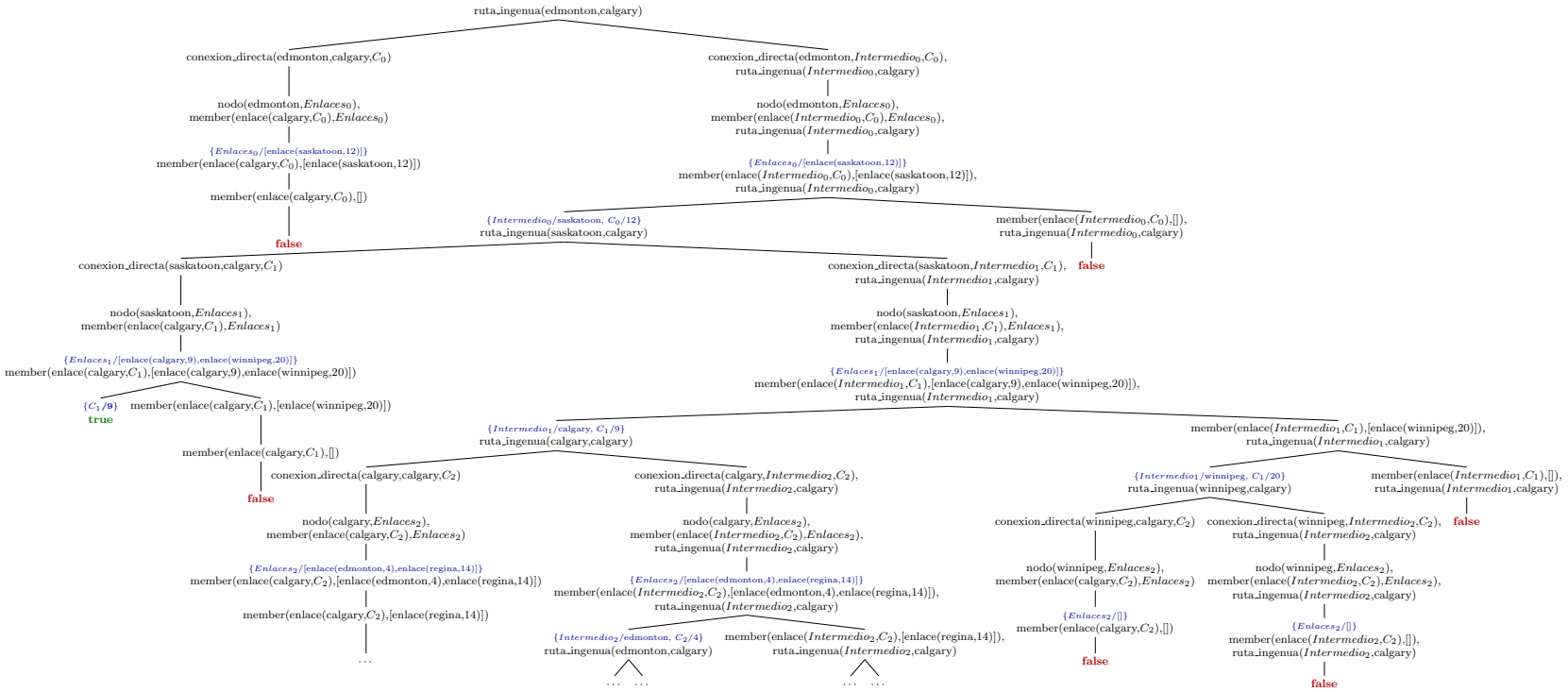


Figura 3.2: Árbol SLD de `ruta_ingenua(edmonton, calgary)` (13 primeros niveles de un árbol infinito).

Capítulo 4

Repositorio

El código fuente de esta actividad y el de las demás actividades del semestre está disponible en GitHub; el historial de versiones del repositorio sirve como soporte del trabajo realizado:

`https://github.com/1594-adrs/ProgIIIG1-Andres_David_Rincon_Salazar`